

Разработка AI-сервиса техподдержки

Выполнил:

студент 4 курса

09.03.01 Информатика и вычислительная
техника, Технологии разработки программного
обеспечения

Гневнов Артем Евгеньевич

Руководитель:

кандидат физико-математических наук, доцент
кафедры ИТиЭО Жуков Николай Николаевич

Актуальность

- Рост обращений в техподдержку → ручная обработка не справляется с пиковой нагрузкой
- Ни одно готовое решение не сочетает: локальное развёртывание + гибкая интеграция + бесплатная лицензия
- Потребность: локальный AI-сервис без передачи данных, встраиваемый в систему управления заявками

Предмет и объект

ОБЪЕКТ ИССЛЕДОВАНИЯ:

Процесс автоматизированной обработки обращений пользователей в системе технической поддержки.

ПРЕДМЕТ ИССЛЕДОВАНИЯ:

Программные методы разработки AI-сервиса технической поддержки в микросервисной архитектуре с применением локальных языковых моделей.

Цель

Разработать микросервис AI-технической поддержки, обеспечивающий:

- автоматическую обработку пользовательских обращений на основе локальной языковой модели;
- сохранение полной истории диалога;
- ограничение нагрузки на языковую модель;
- интеграцию в систему управления заявками.

Задачи

1. Сравнительный анализ существующих AI-решений в области автоматизации технической поддержки.
2. Формулировка функциональных и нефункциональных требований к разрабатываемому AI-сервису.
3. Проектирование архитектуры микросервиса helperbot-ai и протокола его интеграции с системой управления заявками.
4. Реализация микросервиса helperbot-ai: gRPC-интерфейс, хранение чатов в PostgreSQL, вызов локальной LLM через Ollama.
5. Интеграция helperbot-ai с сервисом бизнес-логики helperbot-core и Telegram-ботом helperbot-tg.
6. Тестирование системы и описание процедуры развёртывания через Docker Compose.

ЗАДАЧА 1. АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ

Решения	Развёртывание	Передача данных вовне	Контекст диалога	Интеграция с тикетами	Стоимость	Лицензия
OpenAI Assistants	Облако	Да	Встроенная	Через API	Платно (по токенам)	Проприетарная
Yandex GPT	Облако	Да	Встроенная	Через API	Платно (по токенам)	Проприетарная
Rasa	Локально	Нет	Требуется настройка	Кастомные коннекторы	Бесплатно / Enterprise	Apache 2.0
Botpress	Облако / Локально	Частично	Встроенная	Кастомные плагины	Freemium	Проприетарная
DialogFlow CX	Облако	Да	Встроенная	Через API	Платно	Проприетарная
Ollama custom	+ Локально	Нет	Реализуется разработчиком	Полная	Бесплатно	MIT

ЗАДАЧА 2. Требования к AI-сервису

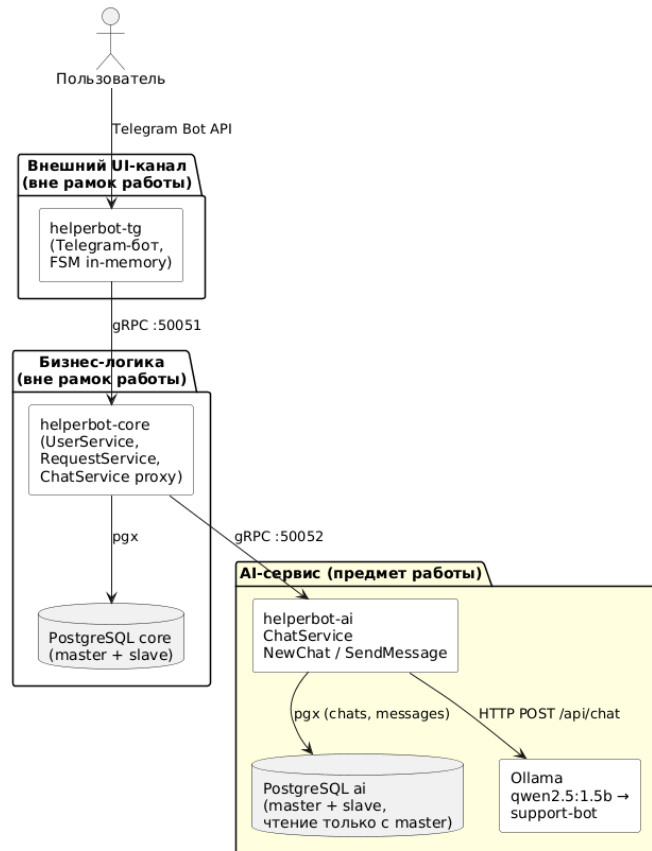
Функциональные требования (6)	Нефункциональные требования (8)
Создание диалога с возвратом UUID чата	Изолированный микросервис с собственной БД
Отправка сообщений с сохранением полного контекста	gRPC + Protocol Buffers, валидация на границе сервиса
Rate limit: не более 10 сообщений / 300 сек	PostgreSQL мастер-реплика
Типизированная ошибка превышения лимита (errors.Is)	Локальное выполнение LLM, без передачи данных вовне
Эскалация к оператору через системный промпт	Таймаут LLM — 60 сек
Встраивание в произвольную систему через gRPC-контракт	Макс. сообщение — 2 000 симв.
	UUID v4 для всех сущностей
	Развёртывание через Docker Compose

ЗАДАЧА 3: Архитектура и интеграционный контракт

Метод	Инициатор	Исполнитель	Назначение
NewChat	helperbot-core	helperbot-ai	Создать новый диалог
SendMessage	helperbot-core	helperbot-ai	Отправить сообщение, получить ответ LLM

Свойства контракта:

- Минимальность — только `user_id`, `chat_id`, `content`
- Декларативная валидация — правила прямо в `.proto`
- Типизированные ошибки — `ErrTooManyRequests` через `%w`



Задача 4: Реализация микросервиса

Компонент	Технология	Обоснование выбора
Язык бэкенда	Go 1.24+	Горутины, статический бинарник, нативный gRPC
Протокол	gRPC + Protocol Buffers	Типизированные контракты, HTTP/2
Валидация	protoc-gen-validate	Декларативные правила валидации в .proto-файле
СУБД	PostgreSQL 16	ACID, UUID, гибкое индексирование
Драйвер БД	pgx v5 + pgxpool	Нативный Go-драйвер, пул соединений
DI-фреймворк	Uber FX	Декларативное внедрение зависимостей
LLM-среда	Ollama	Локальный запуск, HTTP API, Modelfile
Языковая модель	qwen2.5:1.5b	Компактность, поддержка русского языка
Оркестрация	Docker Compose	Единый запуск всего комплекса

Задача 5: Интеграция компонентов

helperbot-tg → helperbot-core → helperbot-ai → Ollama

Состояния FSM в helperbot-tg:

Состояние	Поведение при получении сообщения
PendingActionGetAiMessage	Сообщение отправляется в AI, состояние → PendingActionWaitAiAnswer
PendingActionWaitAiAnswer	Пользователю сообщается, что запрос обрабатывается

Ключевые аспекты интеграции:

- helperbot-core — тонкий прокси, инкапсулирует существование AI-сервиса
- Ленивое создание чата — ChatID запрашивается при первом обращении
- Ошибка лимита — распознаётся через errors.Is, без разбора текста
- Роли пользователей — понятие системы-хоста, AI-сервис их не знает

Задача 6: Тестирование и развёртывание

Юнит-тесты (все PASS):

Тест	Пакет	Что проверяется
TestNewChatID_Valid/Empty/Invalid UUID	model	Конструкторы ChatID
TestNewUserID_Valid/Empty/Invalid UUID	model	Конструкторы UserID
TestSendMessage_RateLimitExceeded	service/chat/v1	ErrTooManyRequests
TestSendMessage_Success	service/chat/v1	Полный успешный сценарий
TestSendMessage_OllamaError	service/chat/v1	Ошибка API-репозитория
TestSendMessage_ContextPassedToOllama	service/chat/v1	Передача контекста в LLM

Команды управления (Makefile):

Команда	Действие
make compose-up	Запустить весь комплекс, создать модель в Ollama
make compose-stop	Остановить контейнеры
make migrate-up-ai	Применить миграции helperbot-ai
make update-repo	Обновить все репозитории (git pull)

Демонстрация работы продукта

